

Bài toán cặp ghép

Hoàng Nam Dũng

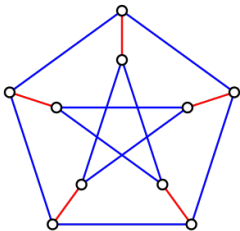
Khoa Toán - Cơ - Tin học, Đại học Khoa học Tự nhiên, Đại học Quốc gia Hà Nội

Bài toán cặp ghép (general matching problem)

Cho một đồ thị vô hướng $G = (N, E)$ với trọng số cạnh c . Một *cặp ghép* (*matching*) trong G là một tập con của E gồm các cạnh đôi một không kề nhau.

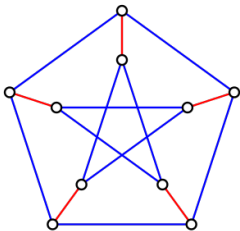
Bài toán cặp ghép (general matching problem)

Cho một đồ thị vô hướng $G = (N, E)$ với trọng số cạnh c . Một *cặp ghép* (*matching*) trong G là một tập con của E gồm các cạnh đôi một không kề nhau.



Bài toán cặp ghép (general matching problem)

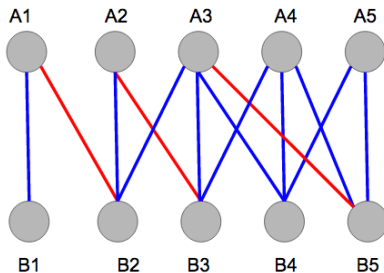
Cho một đồ thị vô hướng $G = (N, E)$ với trọng số cạnh c . Một *cặp ghép* (*matching*) trong G là một tập con của E gồm các cạnh đôi một không kề nhau.



Bài toán cặp ghép tổng quát: Tìm cặp ghép có trọng số cực tiểu.

Các loại bài toán ghép cặp - Ghép cặp hai phía

Ghép cặp hai phía (bipartite matching): G là đồ thị hai phía.

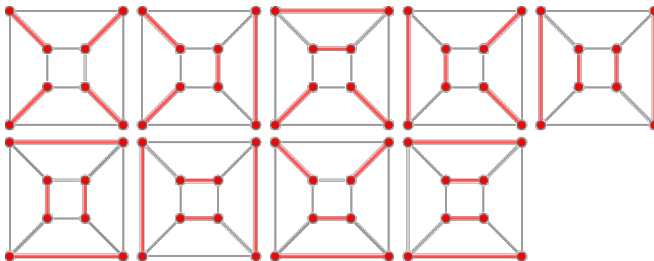


Các loại bài toán ghép cặp - Ghép cặp lực lượng cực đại

Ghép cặp lực lượng cực đại (cardinality matching): Hàm trọng số c là hằng số và bằng -1 . Khi đó cặp ghép có trọng số cực tiểu chính là cặp ghép có lực lượng (số cạnh) cực đại.

Các loại bài toán ghép cặp - Ghép cặp hoàn hảo

Ghép cặp hoàn hảo (perfect matching): Cặp ghép phải thỏa mãn điều kiện mỗi đỉnh phải kề với một cạnh trong cặp ghép, tức là cặp ghép phải có $\frac{|V|}{2}$ cạnh.



Biến đổi tương đương

Ta có thể biến đổi bài toán tìm cặp ghép hoàn hảo có trọng số cực tiểu về bài ghép cặp (một phần) có trọng số cực tiểu

Biến đổi tương đương

Ta có thể biến đổi bài toán tìm cặp ghép hoàn hảo có trọng số cực tiểu về bài ghép cặp (một phần) có trọng số cực tiểu bằng cách giảm trọng số các cạnh một số M đủ lớn, ví dụ có thể chọn $M = 1 + \sum_{e \in E} |c(e)|$ (bài tập).

Biến đổi tương đương

Ta có thể biến đổi bài toán tìm cặp ghép hoàn hảo có trọng số cực tiểu về bài ghép cặp (một phần) có trọng số cực tiểu bằng cách giảm trọng số các cạnh một số M đủ lớn, ví dụ có thể chọn $M = 1 + \sum_{e \in E} |c(e)|$ (bài tập).

Ngược lại ta cũng có thể đưa bài toán ghép cặp có trọng số cực tiểu về bài toán ghép cặp hoàn hảo có trọng số cực tiểu bằng cách thêm đỉnh và cạnh có trọng số thích hợp

Biến đổi tương đương

Ta có thể biến đổi bài toán tìm cặp ghép hoàn hảo có trọng số cực tiểu về bài ghép cặp (một phần) có trọng số cực tiểu bằng cách giảm trọng số các cạnh một số M đủ lớn, ví dụ có thể chọn $M = 1 + \sum_{e \in E} |c(e)|$ (bài tập).

Ngược lại ta cũng có thể đưa bài toán ghép cặp có trọng số cực tiểu về bài toán ghép cặp hoàn hảo có trọng số cực tiểu bằng cách thêm đỉnh và cạnh có trọng số thích hợp

- ▶ Thêm $|V|$ đỉnh giả tạo thành tập đỉnh mới V'
- ▶ Xây dựng đồ thị đầy đủ G' trên V' với trọng số c' được định nghĩa như sau
 - $c'(e) = c(e)$ với mọi $e \in E$, tức là e là cạnh đồ thị ban đầu.
 - $c'(e) = 0$ cho các cạnh còn lại.

Biến đổi tương đương

Ta có thể biến đổi bài toán tìm cặp ghép hoàn hảo có trọng số cực tiểu về bài ghép cặp (một phần) có trọng số cực tiểu bằng cách giảm trọng số các cạnh một số M đủ lớn, ví dụ có thể chọn $M = 1 + \sum_{e \in E} |c(e)|$ (bài tập).

Ngược lại ta cũng có thể đưa bài toán ghép cặp có trọng số cực tiểu về bài toán ghép cặp hoàn hảo có trọng số cực tiểu bằng cách thêm đỉnh và cạnh có trọng số thích hợp

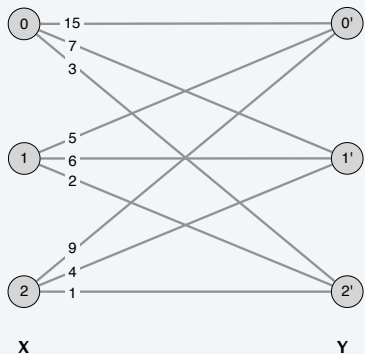
- ▶ Thêm $|V|$ đỉnh giả tạo thành tập đỉnh mới V'
- ▶ Xây dựng đồ thị đầy đủ G' trên V' với trọng số c' được định nghĩa như sau
 - $c'(e) = c(e)$ với mọi $e \in E$, tức là e là cạnh đồ thị ban đầu.
 - $c'(e) = 0$ cho các cạnh còn lại.

Một cặp ghép hoàn hảo có trọng số cực tiểu trên G' là một cặp ghép có trọng số cực tiểu trên G sau khi bỏ đi các cạnh thêm vào.

Assignment problem (bài toán phân công)

Input. Weighted, complete bipartite graph (đồ thị hai phía) $G = (X \cup Y, E)$ with $|X| = |Y|$.

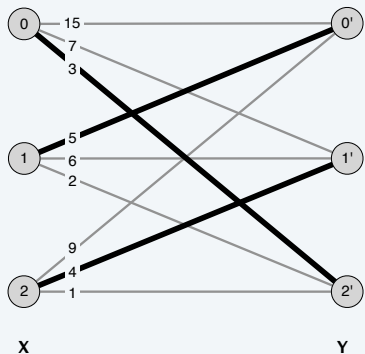
Goal. Find a perfect matching of min weight.



Assignment problem (bài toán phân công)

Input. Weighted, complete bipartite graph (đồ thị hai phía) $G = (X \cup Y, E)$ with $|X| = |Y|$.

Goal. Find a perfect matching of min weight.



min-cost perfect matching

$$M = \{ 0-2', 1-0', 2-1' \}$$

$$\text{cost}(M) = 3 + 5 + 4 = 12$$

Princeton writing seminars

Goal. Given m seminars and $n = 12m$ students who rank their top 8 choices, assign each student to one seminar so that:

- Each seminar is assigned exactly 12 students.
- Students tend to be “happy” with their assigned seminar.

Solution.

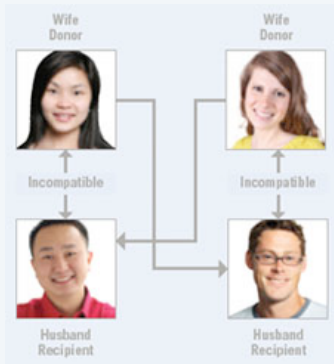
- Create one node for each student i and 12 nodes for each seminar j .
- Solve assignment problem where c_{ij} is some function of the ranks:

$$c_{ij} = \begin{cases} f(\text{rank}(i, j)) & \text{if } i \text{ ranks } j \\ \infty & \text{if } i \text{ does not rank } j \end{cases}$$

Title	Course #	Professor	Day/Time	Location
1980s, The	WRI 168	Scott, Andrea	M/W 1:30pm-2:50pm	Hargadon G002
America and the Melting Pot	WRI 157	Skinazi, Karen	T/TH 8:30am-9:50am	Butler 026
America and the Melting Pot	WRI 158	Skinazi, Karen	T/TH 11:00am-12:20pm	Hargadon G004
American Mysticism	WRI 191	Laufenberg, George	T/TH 7:30pm-8:50pm	99 Alexander 101
American Revolutions	WRI 184	Grosghal, Dov	M/W 8:30am-9:50am	Butler 026
Animal Mind, The	WRI 101	Gould, James	M/W 8:30am-9:50am	Blair T3
Art of Adventure, The	WRI 151	Moffitt, Anne	T/TH 11:00am-12:20pm	Butler 027

Kidney exchange

If a donor and recipient have a different blood type, they can exchange their kidneys with another donor and recipient pair in a similar situation.



Kidney exchange

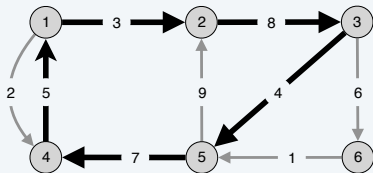
If a donor and recipient have a different blood type, they can exchange their kidneys with another donor and recipient pair in a similar situation.

Can also be done among multiple pairs (or starting with an altruistic donor).

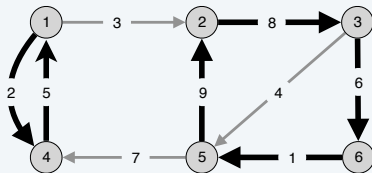


Kidney exchange

See, e.g., <https://blogs.sas.com/content/operations/2015/02/06/the-kidney-exchange-problem/>



$$\text{weight} = 3 + 5 + 7 + 8 + 4 = 27$$



$$\text{weight} = 2 + 5 + 8 + 6 + 1 + 9 = 31$$

Applications

Natural applications.

- Match jobs to machines.
- Match personnel to tasks.
- Match PU students to writing seminars.

Non-obvious applications.

- Vehicle routing.
- Kidney exchange.
- Signal processing.
- Earth-mover's distance.
- Multiple object tracking.
- Virtual output queueing.
- Handwriting recognition.
- Locating objects in space.
- Approximate string matching.
- Enhance accuracy of solving linear systems of equations.

7. MATCHING PROBLEM

- ▶ *maximal cardinality bipartite matching*
- ▶ *assignment problem*
- ▶ *weighted bipartite matching*
- ▶ *general matching*

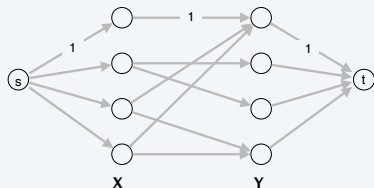
Bipartite matching

Bipartite matching. Can solve via reduction to maximum flow.

Flow. During Ford–Fulkerson, all residual capacities and flows are 0–1; flow corresponds to edges in a matching M .

Residual graph G_M simplifies to:

- If $(x, y) \notin M$, then (x, y) is in G_M .
- If $(x, y) \in M$, then (y, x) is in G_M .



Augmenting path simplifies to:

- Edge from s to an unmatched node $x \in X$,
- Alternating sequence of unmatched and matched edges,
- Edge from unmatched node $y \in Y$ to t .

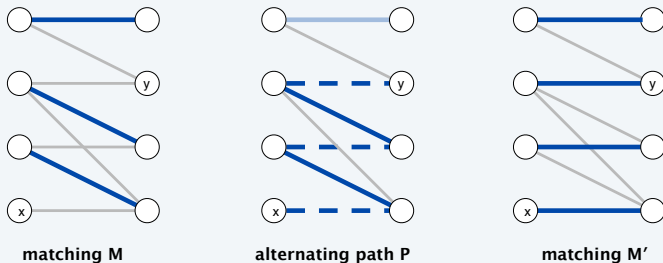
Alternating path (đường thay phiên)

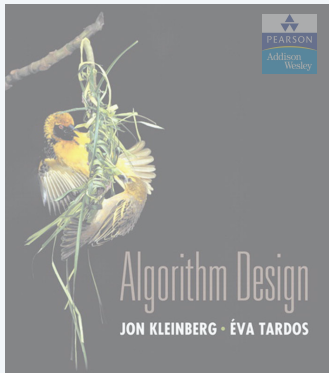
Def. An **alternating path** P with respect to a matching M is an alternating sequence of unmatched and matched edges, starting from an unmatched node $x \in X$ and going to an unmatched node $y \in Y$.

Key property. Can use P to increase by one the cardinality of the matching.

Pf. Set $M' = M \oplus P$.

symmetric difference





SECTION 7.13

7. MATCHING PROBLEM

- ▶ *maximal cardinality bipartite matching*
- ▶ *assignment problem*
- ▶ *weighted bipartite matching*
- ▶ *general matching*

History

Thorndike 1950. Formulated in a modern way by a psychologist.

PSYCHOMETRIKA—VOL. 15, NO. 3
SEPTEMBER, 1950

THE PROBLEM OF CLASSIFICATION OF PERSONNEL*

ROBERT L. THORNDIKE

TEACHERS COLLEGE, COLUMBIA UNIVERSITY

The personnel classification problem arises in its pure form when all job applicants must be used, being divided among a number of job categories. The use of tests for classification involves problems of two types: (1) problems concerning the design, choice, and weighting of tests into a battery, and (2) problems of establishing the optimum administrative procedure of using test results for assignment. A consideration of the first problem emphasizes the desirability of using simple, factorially pure tests which may be expected to have a wide range of validities for different job categories. In the use of test results for assignment, an initial problem is that of expressing predictions of success in different jobs in comparable score units. These units should take account of predictor validity and of job importance. Procedures are described for handling assignment either in terms of daily quotas or in terms of a stable predicted yield.

Assign individuals to jobs to maximize average success of all individuals.

History

Thorndike 1950. Formulated in a modern way by a psychologist.

There are, as has been indicated, a finite number of permutations in the assignment of men to jobs. When the classification problem as formulated above was presented to a mathematician, he pointed to this fact and said that from the point of view of the mathematician there was no problem. Since the number of permutations was finite, one had only to try them all and choose the best. He dismissed the problem at that point. This is rather cold comfort to the psychologist, however, when one considers that only ten men and ten jobs mean over three and a half million permutations. Trying out all the permutations may be a mathematical solution to the problem, it is not a practical solution.

anticipated theory of computational complexity!

History

Kuhn 1955. First poly-time algorithm; named “Hungarian” algorithm to honor two Hungarian mathematicians (Kőnig and Egerváry).

Munkres 1957. Reviewed algorithm; observed $O(n^4)$ implementation.

Edmonds–Karp, Tomizawa 1971. Improved to $O(n^3)$.

THE HUNGARIAN METHOD FOR THE ASSIGNMENT PROBLEM¹

H. W. Kuhn
Bryn Mawr College

Assuming that numerical scores are available for the performance of each of n persons on each of n jobs, the “assignment problem” is the quest for an assignment of persons to jobs so that the sum of the n scores so obtained is as large as possible. It is shown that ideas latent in the work of two Hungarian mathematicians may be exploited to yield a new method of solving this problem.

anticipated development of combinatorial optimization

History

Jacobi (1804–1851). Introduces a bound on the order of a system of m ordinary differential equations in m unknowns and reduces it to....

De investigando ordine systematis aequationum differentialium vulgarium cujuscunque.

(Ex. ill. C. G. J. Jacobi manuscriptis posthumis in medium protulit*) C. W. Borchardt.)

1.

Investigatio ad solvendum problema inaequalitatem reducitur.

Systema aequationum differentialium vulgarium est *non canonicum***, si aequationes altissima variabilium dependentium differentialia tali modo continent, ut horum valores ex iis petere non liceat. Id quod fit, quoties aequationes nonnullae altissimis illis differentialibus carentes in systemate proposito vel ipsae inveniuntur vel eliminatione ex eo obtinentur. *Eo casu numerus Constantium Arbitrariarum, quas integratio completa inducit, sive ordo systematis semper minor est summa altissimorum ordinum, ad quos differentialia singularum variabilium in aequationibus differentialibus propositis ascendunt.* Qui ordo systematis cognoscitur, si per differentiationes et eliminationes contingit systema propositum redigere in aliud forma canonica gaudens eique aequivalens, ita ut de systemate canonico etiam ad propositum reditus pateat. Nam summa altissimorum ordinum, ad quos in systemate canonico differentialia singularum variabilium dependentium ascendunt, etiam systematis propositi non canonici ordo erit. Ad quem ordinem investigandum non tamen opus est ea ad formam canonicam reductione, sed res per considerationes sequentes absolvi potest.

Ponamus inter variabilem independentem t atque n variables dependentes $x_1, x_2, \dots, x_n$ haberi n aequationes differentiales:

$$(1.) \quad u_1 = 0, \quad u_2 = 0, \quad \dots \quad u_n = 0,$$

sitque

$$h_1^{(0)}$$

altissimus ordo, ad quem in aequatione $u_i = 0$ differentialia variabilis x_i ascen-

Looking for the order of a system of arbitrary ordinary differential equations

History

Jacobi (1804–1851). The assignment problem! Moreover, he provides a polynomial-time algorithm.

Problema.

Disponantur nn quantitates $h_k^{(i)}$ quaecunque in schema Quadrati, ita ut habeantur n series horizontales et n series verticales quarum quaeque est n terminorum. Ex illis quantitatibus eligantur n transversales i. e. in seriebus horizontalibus simul atque verticalibus diecris positae, quod fieri potest $1.2 \dots n$ modis; ex omnibus illis modis quaerendus est is qui summam n numerorum electorum suppetit maximam.

Dispositis quantitatibus $h_k^{(i)}$ in figuram quadraticam

$$\begin{array}{cccc} h_1' & h_1'' & \dots & h_n' \\ h_1'' & h_2'' & \dots & h_n'' \\ \dots & \dots & \dots & \dots \\ h_1^{(n)} & h_2^{(n)} & \dots & h_n^{(n)} \end{array}$$

earum systema appellabo *schema propositum*; omne schema inde ortum addendo singulis ejusdem seriei horizontalis terminis eandem quantitatem appellabo *schema derivatum*. Sit

$$f^{(0)}$$

quantitas addenda terminis $f^{(r)}$ seriei horizontalis, quo facto singula $1.2 \dots n$ aggregata transversalia, inter quae maximum eligendum est, eandem augebuntur quantitate

$$f' + f'' + \dots + f^{(n)} = L,$$

quippe ad singula aggregata formanda e quaque serie horizontali unus eligendus est terminus. Qua de re si statuitur

$$h_1^{(0)} + f^{(0)} = p_1^{(0)}$$

atque aggregatum transversale maximum e terminis $h_k^{(0)}$ formatum

$$h_1^{(0)} + h_2^{(0)} + \dots + h_n^{(0)} = H,$$

fit valor aggregati transversalis maximi e terminis $p_k^{(0)}$ formati

$$p_1^{(0)} + p_2^{(0)} + \dots + p_n^{(0)} = H + L$$

Problem.

We dispose nn arbitrary quantities $h_k^{(i)}$ in a square table in such a way that we have n horizontal series and n vertical series having each one n terms. Among these quantities, to chose n being transversal, that is all disposed in different horizontal and vertical series, which may be done in $1.2 \dots n$ ways; and among these ways, to research one that gives the maximum of the sum of the n chosen numbers.

$$\begin{array}{cccc} h_1' & h_2' & \dots & h_n' \\ h_1'' & h_2'' & \dots & h_n'' \\ \dots & \dots & \dots & \dots \\ h_1^{(n)} & h_2^{(n)} & \dots & h_n^{(n)} \end{array}$$

we can add to each term of the same horizontal series a same quantity, and we call $\ell^{(i)}$ the quantity added to the terms of the i^{th} horizontal series. This being done, each of the $1.2 \dots n$ transversal sums among which we need to find a maximum is increased by the same quantity

$$\ell' + \ell'' + \dots + \ell^{(n)} = L,$$

because, in order to form these sums, we need to pick a term in each horizontal series. Hence, if we pose

$$h_k^{(i)} + \ell^{(i)} = p_k^{(i)}$$

and that the maximal transversal sum of the terms $h_k^{(0)}$ is

$$h_1^{(0)} + h_2^{(0)} + \dots + h_n^{(0)} = H,$$

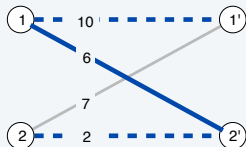
this makes that the value of the maximal sum formed with the $p_k^{(i)}$ is

$$p_1^{(i)} + p_2^{(i)} + \dots + p_n^{(i)} = H + L,$$

Jacobi formulated the assignment problem; proposed and analyzed the Hungarian algorithm

Assignment problem: successive shortest path algorithm

Cost of alternating path. Pay $c(x, y)$ to match $x-y$; receive $c(x, y)$ to unmatched.



$$P = 2 \rightarrow 2' \rightarrow 1 \rightarrow 1'$$

$$\text{cost}(P) = 2 - 6 + 10 = 6$$

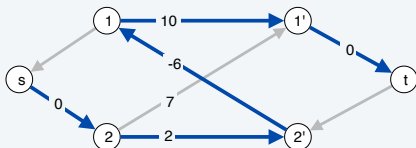
Shortest alternating path. Alternating path from any unmatched node $x \in X$ to any unmatched node $y \in Y$ with smallest cost.

Successive shortest path algorithm (thuật toán đường đi ngắn nhất liên tiếp).

- Start with empty matching.
- Repeatedly augment along a **shortest** alternating path.

Finding the shortest alternating path

Shortest alternating path. Corresponds to minimum cost $s \rightarrow t$ path in G_M .



Concern. Edge costs can be negative.

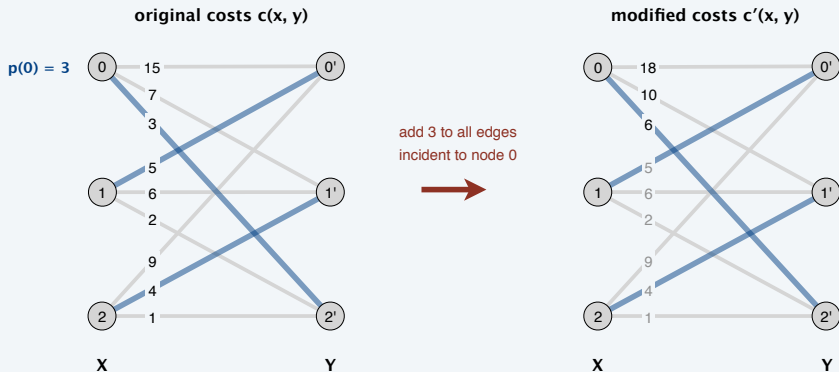
Fact. If always choose shortest alternating path, then G_M contains no negative cycles $\Rightarrow$ can compute using Bellman-Ford.

Our plan. Use **duality** to avoid negative edge costs (and negative cycles)
 $\Rightarrow$ can compute using Dijkstra.

Equivalent assignment problem

Duality intuition. Adding a constant $p(x)$ to the cost of every edge incident to node $x \in X$ does not change the min-cost perfect matching(s).

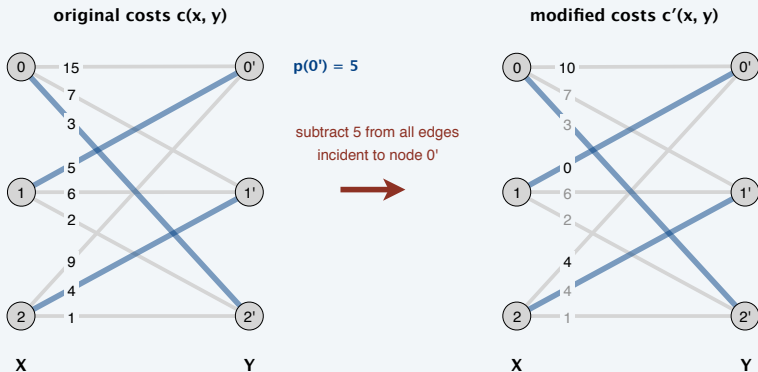
Pf. Every perfect matching uses exactly one edge incident to node x . ■



Equivalent assignment problem

Duality intuition. Subtracting a constant $p(y)$ to the cost of every edge incident to node $y \in Y$ does not change the min-cost perfect matching(s).

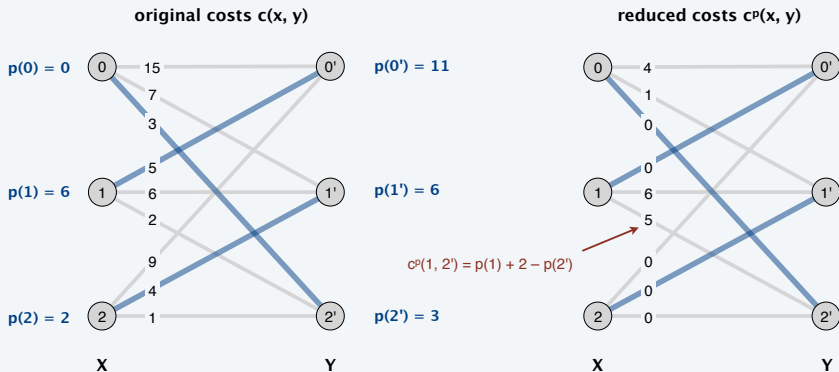
Pf. Every perfect matching uses exactly one edge incident to node y . ▀



Reduced costs (chi phí rút gọn)

Reduced costs. For $x \in X, y \in Y$, define $c^p(x, y) = p(x) + c(x, y) - p(y)$.

Observation 1. Finding a min-cost perfect matching with reduced costs is equivalent to finding a min-cost perfect matching with original costs.



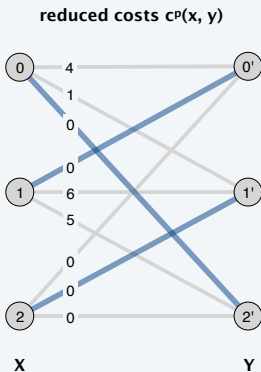
Compatible prices (giá tương thích)

Compatible prices. For each node $v \in X \cup Y$, maintain prices $p(v)$ such that:

- $c^p(x, y) \geq 0$ for all $(x, y) \notin M$.
- $c^p(x, y) = 0$ for all $(x, y) \in M$.

Observation 2. If prices p are compatible with a **perfect** matching M , then M is a min-cost perfect matching.

Pf. Matching M has 0 cost. ▀



Successive shortest path algorithm (đường đi ngắn nhất liên tiếp)

SUCCESSIVE-SHORTEST-PATH (X, Y, c)

$M \leftarrow \emptyset$.

FOREACH $v \in X \cup Y : p(v) \leftarrow 0$.

┌

prices p are
compatible with M
 $c^p(x, y) = c(x, y) \geq 0$

WHILE (M is not a perfect matching)

$d \leftarrow$ shortest path distances using costs c^p .

$P \leftarrow$ shortest alternating path using costs c^p .

$M \leftarrow$ updated matching after augmenting along P .

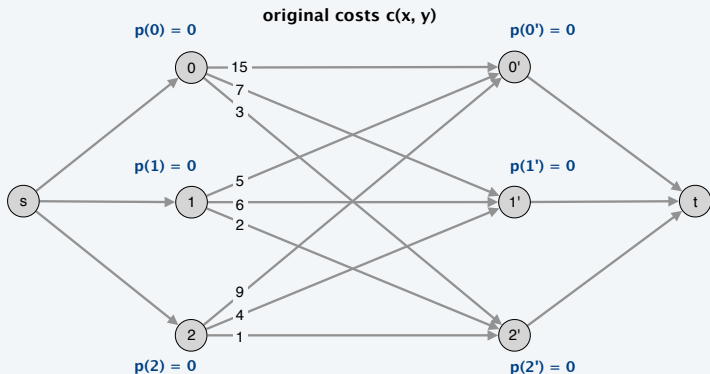
 FOREACH $v \in X \cup Y : p(v) \leftarrow p(v) + d(v)$.

RETURN M .

Successive shortest path algorithm

Initialization.

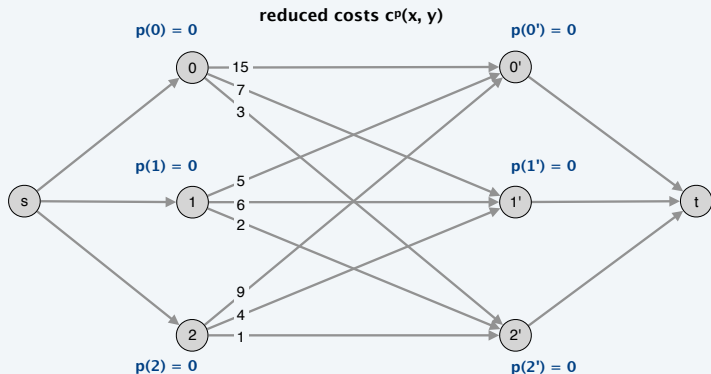
- $M = \emptyset$.
- For each $v \in X \cup Y: p(v) \leftarrow 0$.



Successive shortest path algorithm

Initialization.

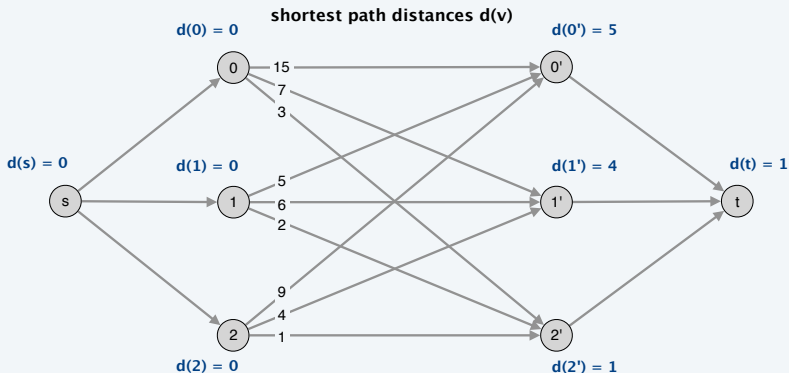
- $M = \emptyset$.
- For each $v \in X \cup Y : p(v) \leftarrow 0$.



Successive shortest path algorithm

Step 1.

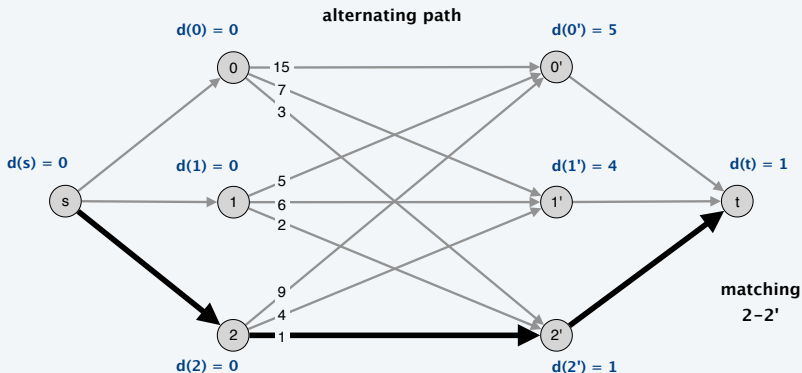
- Compute shortest path distances $d(v)$ from s to v using $cp(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Step 1.

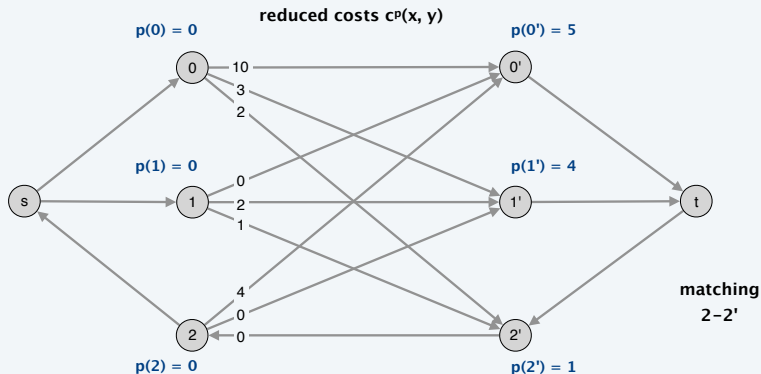
- Compute shortest path distances $d(v)$ from s to v using $cp(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Step 1.

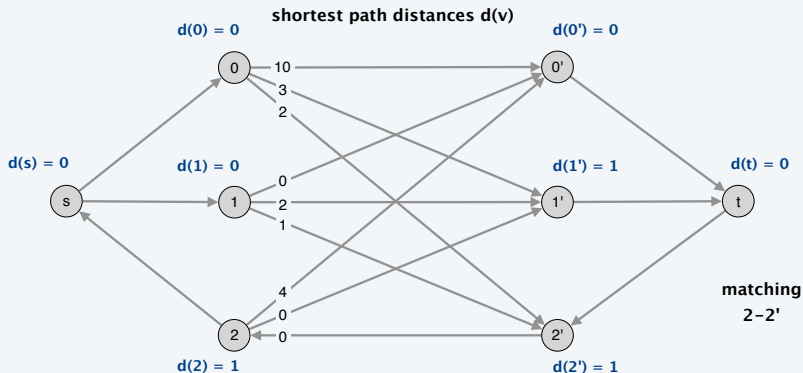
- Compute shortest path distances $d(v)$ from s to v using $c^p(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Step 2.

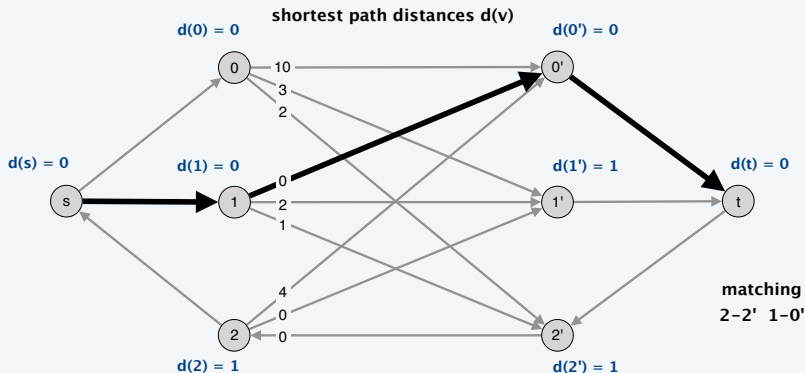
- Compute shortest path distances $d(v)$ from s to v using $cp(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Step 2.

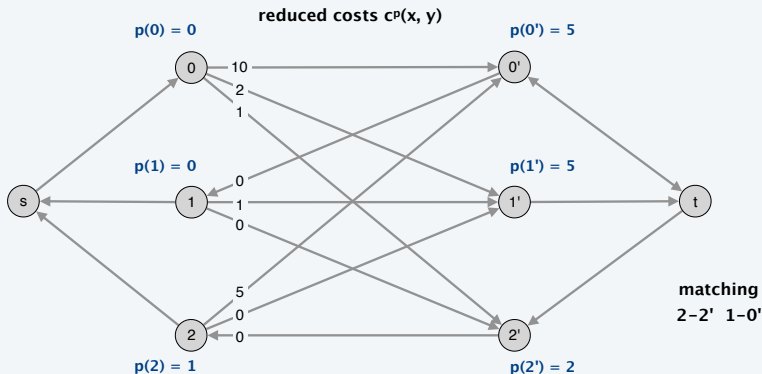
- Compute shortest path distances $d(v)$ from s to v using $cp(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Step 2.

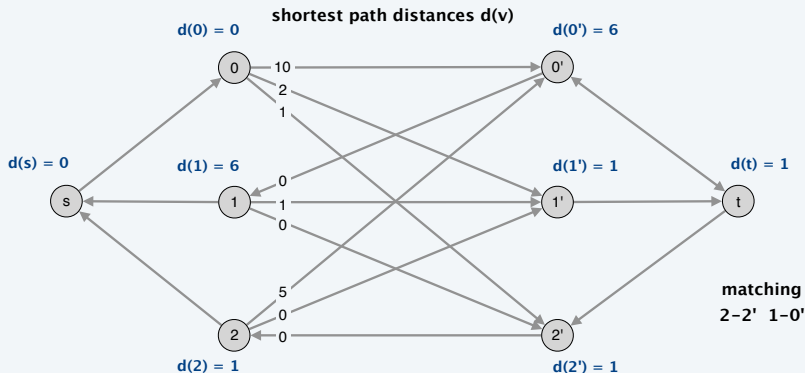
- Compute shortest path distances $d(v)$ from s to v using $c^p(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Step 3.

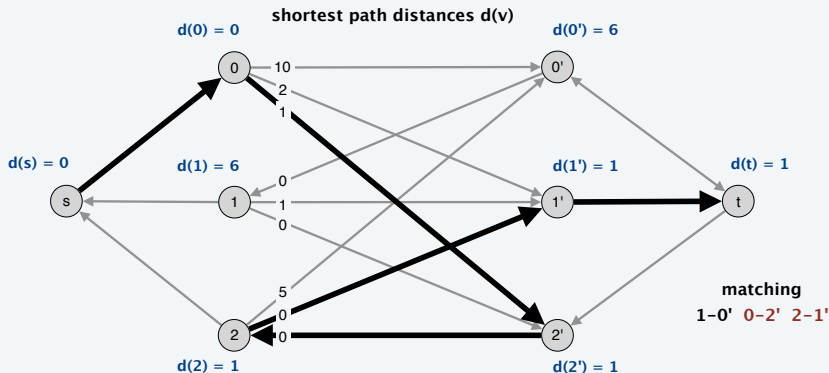
- Compute shortest path distances $d(v)$ from s to v using $cp(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Step 3.

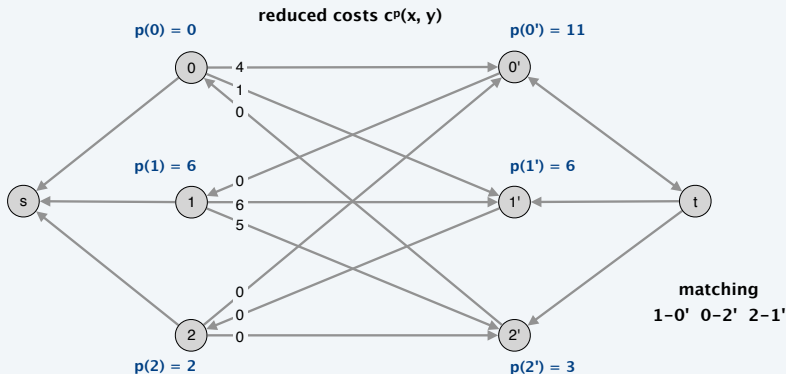
- Compute shortest path distances $d(v)$ from s to v using $cp(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Step 3.

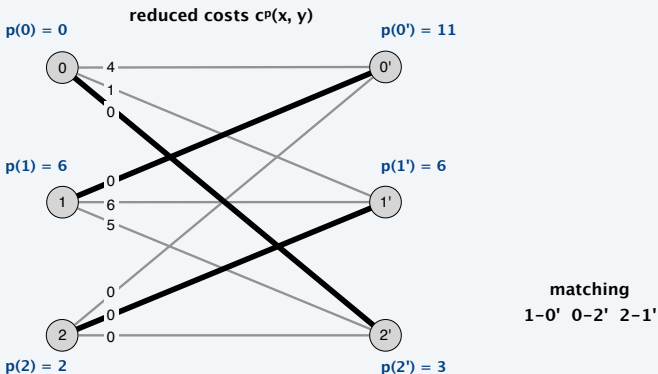
- Compute shortest path distances $d(v)$ from s to v using $c^p(x, y)$.
- Update matching M via shortest path from s to t .
- For each $v \in X \cup Y$: $p(v) \leftarrow p(v) + d(v)$.



Successive shortest path algorithm

Termination.

- M is a perfect matching.
- Prices p are compatible with M .



Maintaining compatible prices (duy trì giá tương thích)

Lemma 1. Let p be compatible prices for M . Let d be shortest path distances in G_M with costs c^p . All edges (x, y) on shortest path have $c^{p+d}(x, y) = 0$.

 forward or reverse edges

Pf. Let (x, y) be some edge on shortest path.

- If $(x, y) \in M$, then (y, x) on shortest path and $d(x) = d(y) - c^p(x, y)$;
If $(x, y) \notin M$, then (x, y) on shortest path and $d(y) = d(x) + c^p(x, y)$.
- In either case, $d(x) + c^p(x, y) - d(y) = 0$.
- By definition, $c^p(x, y) = p(x) + c(x, y) - p(y)$.
- Substituting for $c^p(x, y)$ yields $(p(x) + d(x)) + c(x, y) - (p(y) + d(y)) = 0$.
- In other words, $c^{p+d}(x, y) = 0$. ■

Given prices p , the reduced cost of edge (x, y) is

$$c^p(x, y) = p(x) + c(x, y) - p(y).$$

Maintaining compatible prices

Lemma 2. Let p be compatible prices for M . Let d be shortest path distances in G_M with costs c^p . Then $p' = p + d$ are also compatible prices for M .

Pf. $(x, y) \in M$

- (y, x) is the only edge entering x in G_M . Thus, (y, x) on shortest path.
- By LEMMA 1, $c^{p+d}(x, y) = 0$.

Pf. $(x, y) \notin M$

- (x, y) is an edge in $G_M \Rightarrow d(y) \leq d(x) + c^p(x, y)$.
- Substituting $c^p(x, y) = p(x) + c(x, y) - p(y) \geq 0$ yields
 $(p(x) + d(x)) + c(x, y) - (p(y) + d(y)) \geq 0$.
- In other words, $c^{p+d}(x, y) \geq 0$. ■

Prices p are compatible with matching M :

- $c^p(x, y) \geq 0$ for all $(x, y) \notin M$.
- $c^p(x, y) = 0$ for all $(x, y) \in M$.

Maintaining compatible prices

Lemma 3. Let p be compatible prices for M and let M' be matching obtained by augmenting along a min cost path with respect to c^{p+d} . Then $p' = p + d$ are compatible prices for M' .

Pf.

- By LEMMA 2, the prices $p + d$ are compatible for M .
- Since we augment along a min-cost path, the only edges (x, y) that swap into or out of the matching are on the min-cost path.
- By LEMMA 1, these edges satisfy $c^{p+d}(x, y) = 0$.
- Thus, compatibility is maintained. ■

Prices p are compatible with matching M :

- $c^p(x, y) \geq 0$ for all $(x, y) \notin M$.
- $c^p(x, y) = 0$ for all $(x, y) \in M$.

Successive shortest path algorithm: analysis

Invariant. The algorithm maintains a matching M and compatible prices p .

Pf. Follows from LEMMA 2 and LEMMA 3 and initial choice of prices. ■

Theorem. The algorithm returns a min-cost perfect matching.

Pf. Upon termination M is a perfect matching, and p are compatible prices.

Optimality follows from OBSERVATION 2. ■

Theorem. The algorithm can be implemented in $O(n^3)$ time.

Pf.

- Each iteration increases the cardinality of M by 1 $\Rightarrow n$ iterations.
- Bottleneck operation is computing shortest path distances d .

Since all costs are nonnegative, each iteration takes $O(n^2)$ time using (dense) Dijkstra. ■

7. MATCHING PROBLEM

- ▶ *maximal cardinality bipartite matching*
- ▶ *assignment problem*
- ▶ ***weighted bipartite matching***
- ▶ *general matching*

Weighted bipartite matching

Weighted bipartite matching. Given a weighted bipartite graph with n nodes and m edges, find a maximum cardinality matching of minimum weight.

Theorem. [Fredman–Tarjan 1987] The successive shortest path algorithm solves the problem in $O(n^2 + mn \log n)$ time using Fibonacci heaps.

Theorem. [Gabow–Tarjan 1989] There exists an $O(mn^{1/2} \log(nC))$ time algorithm for the problem when the costs are integers between 0 and C .

SIAM J. COMPUT.
Vol. 18, No. 5, pp. 1013–1036, October 1989

©1989 Society for Industrial and Applied Mathematics
011

FASTER SCALING ALGORITHMS FOR NETWORK PROBLEMS*

HAROLD N. GABOW† AND ROBERT E. TARJAN‡

Abstract. This paper presents algorithms for the assignment problem, the transportation problem, and the minimum-cost flow problem of operations research. The algorithms find a minimum-cost solution, yet run in time close to the best-known bounds for the corresponding problems without costs. For example, the assignment problem (equivalently, minimum-cost matching in a bipartite graph) can be solved in $O(\sqrt{nm} \log(nN))$ time, where n , m , and N denote the number of vertices, number of edges, and largest magnitude of a cost; costs are assumed to be integral. The algorithms work by scaling. As in the work of Goldberg and Tarjan, in each scaled problem an approximate optimum solution is found, rather than an exact optimum.

7. MATCHING PROBLEM

- ▶ *maximal cardinality bipartite matching*
- ▶ *assignment problem*
- ▶ *weighted bipartite matching*
- ▶ *general matching*

Bài toán ghép cặp tổng quát

Các bài toán ghép cặp trên đồ thị tổng quát

- ▶ Bài toán ghép cặp có lực lượng cực đại
- ▶ Bài toán ghép cặp có trọng số cực tiểu

đều có thể giải được trong thời gian đa thức ¹.

¹Bernhard Korte and Jens Vygen, Combinatorial Optimization: Theory and Algorithms, Springer, 2006